

Government College of Engineering, Jalgaon
(An Autonomous Institute of Government of Maharashtra)

Name :

Class : L. Y. B.Tech Computer

Subject : CO456U Cloud Computing Lab

Course Teacher: Prof. S. D. Cheke

Date of Performance :

PRN :

Batch:

Sem : VIIIth

Academic Year : 2025-26

Date of Completion :

Practical No - 5

Aim:- To create an AWS S3 bucket and deploy a static one-page website on it, and demonstrate uploading files to the S3 bucket using a Python script.

Theory:-

Amazon S3 (Simple Storage Service) is an object storage service provided by Amazon Web Services that allows users to store and retrieve data from anywhere on the internet.

S3 supports static website hosting, which allows users to host HTML, CSS, and JavaScript files without using a traditional web server.

Benefits of S3 Static Hosting:

- Highly scalable
- Low cost
- Easy deployment
- Integrated with other AWS services

Python scripts can interact with AWS services using the **AWS SDK for Python (Boto3)** provided by Amazon.

Requirements:-

Software

- AWS Account
- Python 3.x
- AWS CLI installed
- Internet browser

Python Libraries

- boto3

Install using:

```
pip install boto3
```

Architecture Diagram:-

User Browser

|

v

AWS S3 Bucket (Static Website Hosting Enabled)

|

v

HTML / CSS / JS Files

|

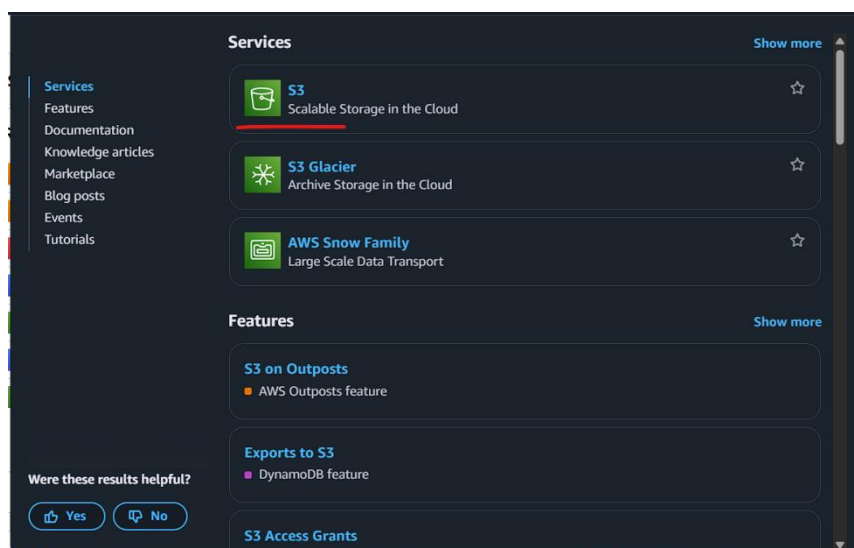
v

Uploaded using Python Script (Boto3)

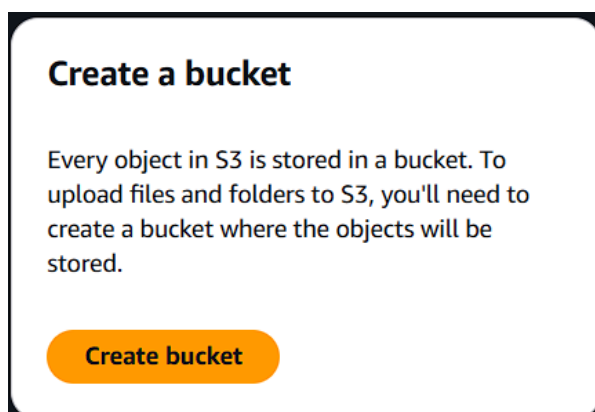
Procedure:-

Part 1: Create an S3 Bucket

1. Login to the AWS Management Console.
2. Open the S3 Service in Amazon Web Services.



3. Click Create Bucket.



4. Enter:
 1. Bucket Name: unique name
Example: my-static-website-bucket123

Bucket name [Info](#)

my-amzn-bucket

Bucket names must be 3 to 63 characters and unique within the global namespace. Bucket names must also begin and end with a letter or number. Valid characters are a-z, 0-9, periods (.), and hyphens (-). [Learn more](#)

5. Disable "Block Public Access" (required for public website hosting).

Block Public Access settings for this bucket

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to this bucket and its objects is blocked, turn on Block all public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to this bucket or objects within, you can customize the individual settings below to suit your specific storage use cases. [Learn more](#)

☐ **Block all public access**
Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.

- ☐ **Block public access to buckets and objects granted through new access control lists (ACLs)**
S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for existing buckets and objects. This setting doesn't change any existing permissions that allow public access to S3 resources using ACLs.
- ☐ **Block public access to buckets and objects granted through any access control lists (ACLs)**
S3 will ignore all ACLs that grant public access to buckets and objects.
- ☐ **Block public access to buckets and objects granted through new public bucket or access point policies**
S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change any existing policies that allow public access to S3 resources.
- ☐ **Block public and cross-account access to buckets and objects through any public bucket or access point policies**
S3 will ignore public and cross-account access for buckets or access points with policies that grant public access to buckets and objects.

Turning off block all public access might result in this bucket and the objects within becoming public
AWS recommends that you turn on block all public access, unless public access is required for specific and verified use cases such as static website hosting.

☐ I acknowledge that the current settings might result in this bucket and the objects within becoming public.

6. Acknowledge the warning.

Turning off block all public access might result in this bucket and the objects within becoming public
AWS recommends that you turn on block all public access, unless public access is required for specific and verified use cases such as static website hosting.

☒ I acknowledge that the current settings might result in this bucket and the objects within becoming public.

7. Click Create Bucket.

[Cancel](#)[Create bucket](#)

Part 2: Enable Static Website Hosting

1. Open the created bucket.
2. Go to **Properties**.

my-amzn-bucket123 [Info](#)

[Objects](#)[Metadata](#)[Properties](#)[Permissions](#)[Metrics](#)[Management](#)[File systems - new](#)[Access Points](#)

3. Scroll to **Static Website Hosting**.

Static website hosting

Use this bucket to host a website or redirect requests. [Learn more](#)

Edit

We recommend using AWS Amplify Hosting for static website hosting
Deploy a fast, secure, and reliable website quickly with AWS Amplify Hosting. Learn more about [Amplify Hosting](#) or [View your existing Amplify apps](#)

Create Amplify app

S3 static website hosting

Disabled

4. Click **Edit**.
5. Select: Enable

In Index document type index.html

Static website hosting

Use this bucket to host a website or redirect requests. [Learn more](#)

Static website hosting

☐ Disable
☒ Enable

Hosting type

☒ Host a static website

Use the bucket endpoint as the web address. [Learn more](#)

☐ Redirect requests for an object

Redirect requests to another bucket or domain. [Learn more](#)

For your customers to access content at the website endpoint, you must make all your content publicly readable. To do so, you can edit the S3 Block Public Access settings for the bucket. For more information, see [Using Amazon S3 Block Public Access](#)

Index document

Specify the home or default page of the website.

index.html

Save Changes

Cancel

Save changes

Part 3: Set Bucket Policy for Public Access

Go to **Permissions** → **Bucket Policy** and paste:

my-amzn-bucket123 [Info](#)

Objects

Metadata

Properties

Permissions

Metrics

Management

File systems - new

Access Points

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": ["s3:GetObject"],
      "Resource": ["arn:aws:s3:::my-static-website-bucket123/*"]
    }
  ]
}
```

Edit bucket policy [Info](#)

Bucket policy

The bucket policy, written in JSON, provides access to the objects stored in the bucket. Bucket policies don't apply to objects owned by other accounts. [Learn more](#)

Bucket ARN

 arn:aws:s3::my-amzn-bucket123

Policy

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Sid": "PublicReadGetObject",  
6       "Effect": "Allow",  
7       "Principal": "*",  
8       "Action": ["s3:GetObject"],  
9       "Resource": ["arn:aws:s3::my-amzn-bucket123/*"]  
10    }  
11  ]  
12 }
```

Replace the bucket name accordingly in “Resource”.

Save the policy.

Cancel

Save changes

Part 4: Create Static Web Page

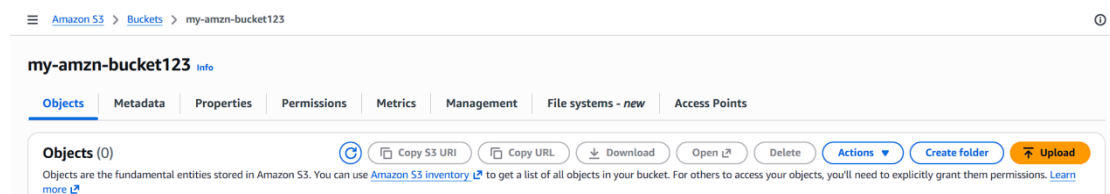
Create a file **index.html**

```
<!DOCTYPE html>  
<html>  
<head>  
<title>My First S3 Website</title>  
</head>  
  
<body>  
  
<h1>Welcome to My Static Website</h1>  
<p>This website is hosted on AWS S3.</p>  
  
</body>  
</html>
```

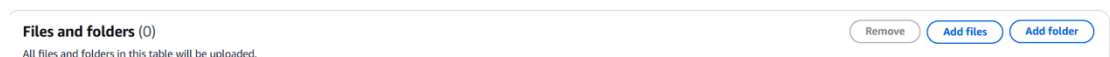
```
index.html > html
1  <!DOCTYPE html>
2  <html>
3  <head>
4  <title>My First S3 Website</title>
5  </head>
6
7  <body>
8
9  <h1>Welcome to My Static Website</h1>
10 <p>This website is hosted on AWS S3.</p>
11
12 </body>
13 </html>
```

Part 5: Upload Files Manually

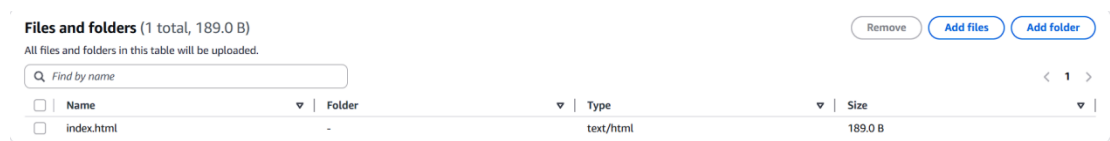
1. Open S3 Bucket.
2. Click **Upload**.



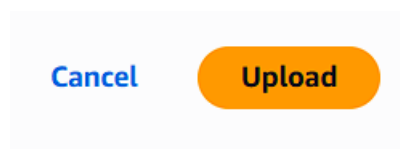
3. Click on “Add files” or “Add folder”



4. Select index.html from your pc



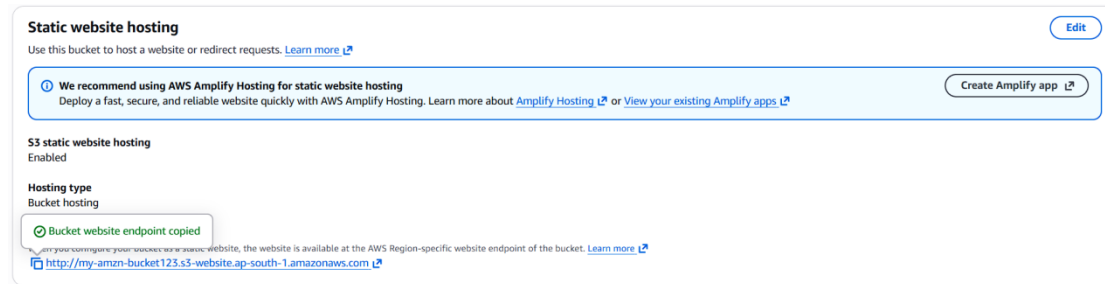
5. Click Upload



Part 6: Access the Website

Go to:

Properties → Static Website Hosting



Copy the **Endpoint URL**.

Example:

`http://my-static-website-bucket123.s3-website-ap-south-1.amazonaws.com`

Open it in the browser.

Your **static website will load**.

Welcome to My Static Website

This website is hosted on AWS S3.

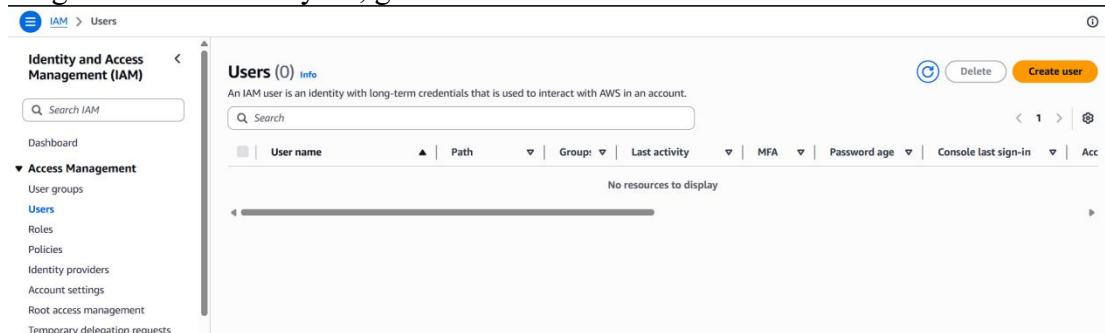
Part 7: Upload Files Using Python Script

Step 1: Install awscli from

`"https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html"`

1. Use `aws --version` to check the version in cmd prompt
2. Type `aws configure` in cmd prompt

To get AWS Access Key ID, go to aws console and search IAM and click on it.



Step 2 :Go to left panel and select Users. Then click “Create User” and type a user name of your choice.

IAM > Users > Create user

Step 1: Specify user details

Specify user details

User details

User name

The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and + = . @ _ - (hyphen)

☐ **Provide user access to the AWS Management Console - optional**
In addition to console access, users with `SigntnlLocalDevelopmentAccess` permissions can use the same console credentials for programmatic access without the need for access keys.

📘 If you are creating programmatic access through access keys or service-specific credentials for AWS CodeCommit or Amazon Keyspaces, you can generate them after you create this IAM user. [Learn more](#)

Cancel Next

Click “Next”.

Step 3: Give Permissions

1. Select **Attach policies directly**.
2. Search and select: `AmazonS3FullAccess`

Click Next → Create user.

IAM > Users > Create user

Step 3: Review and create

Permissions options

☐ **Add user to group**
Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.

☐ **Copy permissions**
Copy all group memberships, attached managed policies, and inline policies from an existing user.

☒ **Attach policies directly**
Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.

Permissions policies (1481)

Choose one or more policies to attach to your new user.

Filter by Type: All types 1 match

Policy name	Type	Attached entities
☐ AmazonS3FullAccess	AWS managed	0

Create policy

User details

User name
s3-user

Console password type
None

Require password reset
No

Permissions summary

Name	Type	Used as
AmazonS3FullAccess	AWS managed	Permissions policy

Tags - optional
 Tags are key-value pairs you can add to AWS resources to help identify, organize, or search for resources. Choose any tags you want to associate with this user.
 No tags associated with the resource.

Add new tag

You can add up to 50 more tags.

Cancel Previous Create user

✕

🟢 User created successfully

You can view and download the user's password and email instructions for signing in to the AWS Management Console.

View user

Users (1) Info

⌂
Delete
Create user

An IAM user is an identity with long-term credentials that is used to interact with AWS in an account.

Q Search
< 1 > ⚙

☐	User name	Path	Group	Last activity	MFA	Password age	Console last sign-in	Acc
☐	s3-user	/	0	-	-	-	-	-

Step 4: Create Access Key

1. Click the user you created.
2. Go to Security Credentials tab.
3. Scroll to Access Keys.
4. Click Create Access Key.

Access keys (0)

Create access key

Use access keys to send programmatic calls to AWS from the AWS CLI, AWS Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time. [Learn more](#)

No access keys. As a best practice, avoid using long-term credentials like access keys. Instead, use tools which provide short term credentials. [Learn more](#)

Create access key

Choose:

Command Line Interface (CLI)

Access key best practices & alternatives Info

Avoid using long-term credentials like access keys to improve your security. Consider the following use cases and alternatives.

Use case

☒

Command Line Interface (CLI)
 You plan to use this access key to enable the AWS CLI to access your AWS account.

☐

Local code
 You plan to use this access key to enable application code in a local development environment to access your AWS account.

☐

Application running on an AWS compute service
 You plan to use this access key to enable application code running on an AWS compute service like Amazon EC2, Amazon ECS, or AWS Lambda to access your AWS account.

☐

Third-party service
 You plan to use this access key to enable access for a third-party application or service that monitors or manages your AWS resources.

Click Next → Create Access Key.

Cancel

Previous

Create access key

Now copy the Access key and Secret Access key in the terminal where it is required.
 Default region name: (shown in the top right corner of your aws console)
 Default output format: json

Step 2: Python Script to Upload File

Create file:

upload.py

Python code:

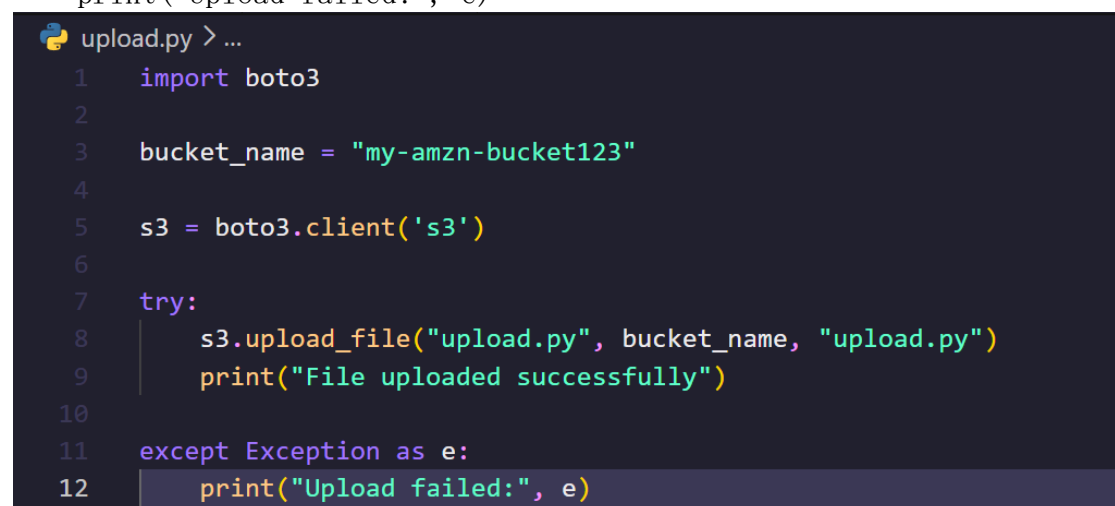
```
import boto3

bucket_name = "my-static-website-bucket123"

s3 = boto3.client('s3')

try:
    s3.upload_file(file_name, bucket_name, "file_name")
    print("File uploaded successfully")

except Exception as e:
    print("Upload failed:", e)
```



```
upload.py > ...
1  import boto3
2
3  bucket_name = "my-amzn-bucket123"
4
5  s3 = boto3.client('s3')
6
7  try:
8      s3.upload_file("upload.py", bucket_name, "upload.py")
9      print("File uploaded successfully")
10
11 except Exception as e:
12     print("Upload failed:", e)
```

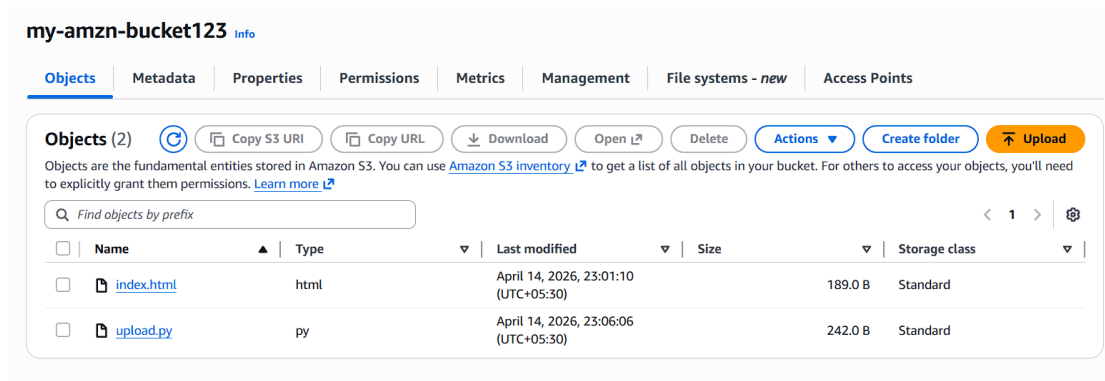
Step 3: Run Script

python upload.py

Output:

File uploaded successfully

The file will now appear inside the **S3 bucket**.



Result:-

The S3 bucket was successfully created, static website hosting was enabled, and the website was deployed and accessed through the S3 endpoint. Files were also uploaded programmatically using a Python script with Boto3.

Conclusion:-

This experiment demonstrates how Amazon S3 can be used as a lightweight web hosting platform. Using Python and Boto3, developers can automate file uploads and integrate S3 into applications.

Prof. S. D. Cheke
Course Teacher